Shoof App Architecture Plan

Prepared by Mohammad Muhaisen, 18 October 2026

Overview:

Shoof is a short-video app where **business owners** post product reels, and **sale reels** go live on a chosen start date and disappear automatically when the sale ends.

Users: business owners post reels; customers watch reels, follow businesses, like, comment and message them. Both have accounts.

Platforms: Android and iOS from one Flutter codebase.

Main feature: a sale reel has a start and end time (for example, publish tomorrow and keep it live for 7 days). Regular reels stay permanently.

Customers in the MVP: watch, follow, like, comment and send direct messages to business owners. Buying is agreed in the messages, outside the app.

MVP scope:

The first version covers eight things; everything else waits.

Build now

1) Accounts for owners and customers (sign up, login, logout)
2) Business profile for owners (name, bio, contact number)
3) Upload a reel (video, caption, product name, price)
4) Two reel types: regular (permanent) and sale (start date, end date)
5) Follow businesses, and a feed of reels from the businesses a customer follows
6) Like and comment on reels
7) Direct messages between customers and business owners, like Instagram DMs

Later

Payments and ordering inside the app, analytics, push notifications (new sale, new message), group chats, and photos or videos inside messages.

Requirements:

These requirements come from the project idea, not from meetings with real users. They should be checked with a few business owners and customers before the app is built.

UR1: Users shall be able to register an account and log in using their email and password, as an owner or a customer.

UR2: Business owners shall be able to create a business profile with a name, bio and contact number.

UR3: Business owners shall be able to upload a reel with a video, caption, product name and price.

UR4: Business owners shall be able to schedule a sale reel with a start date and an end date.

UR5: Registered customers shall be able to follow and unfollow businesses.

UR6: Registered customers shall be able to view a feed of reels from the businesses they follow.

UR7: The system shall hide a sale reel automatically when its sale period is not active.

UR8: Registered customers shall be able to like and comment on reels.

UR9: Customers and business owners shall be able to send direct messages to each other.

System Requirements:

UR1:

SR1.1: The system shall enable users to enter name, email, password and role during registration.

SR1.2: The system shall validate that the email is unique and the password has at least 8 characters.

SR1.3: The system shall give the user an access token after a successful login.

UR2:

SR2.1: The system shall allow only owners to create one business profile.

UR3:

SR3.1: The system shall issue a signed upload request to logged-in owners only.

SR3.2: The system shall save the video URL, thumbnail URL, caption, product name and price after the upload.

SR3.3: The system shall limit reel length and file size.

UR4:

SR4.1: The system shall enable owners to choose regular or sale as the reel type.

SR4.2: The system shall reject a sale reel when the end date is not after the start date.

UR5:

SR5.1: The system shall prevent a customer from following the same business twice.

UR6:

SR6.1: The system shall show a reel only if it is regular, or if it is a sale reel and the current time is between its start and end dates.

SR6.2: The system shall show the newest reels first.

UR7:

SR7.1: The system shall run a scheduler every minute to mark ended sale reels as expired.

UR8:

SR8.1: The system shall allow one like per user on each reel, and let the user remove it.

SR8.2: The system shall enable users to add a comment of up to 300 characters to a reel.

SR8.3: The system shall show the like count and the comments of each reel, newest comments first.

SR8.4: The system shall enable owners to delete comments on their own reels.

UR9:

SR9.1: The system shall enable a customer to start a conversation with a business owner from the business profile.

SR9.2: The system shall enable both sides to send and read text messages in that conversation.

SR9.3: The system shall show a list of conversations with the latest message first.

SR9.4: The system shall allow only the two participants to read a conversation.

Actors:

The actors are the people and outside services that interact with the system.

Actors	Description
Business Owner	This actor represents a business that creates a profile, uploads reels, schedules sale reels and replies to messages from customers.
Registered Customer	This actor represents a user who has an account, follows businesses, views their reels in the feed, likes and comments on them, and sends messages to owners.
Cloudinary	This actor represents an external service that stores the videos and streams them to phones.
Scheduler	This actor represents the timer inside Laravel that marks ended sale reels as expired.
Administration	Later version only. This actor represents staff who remove inappropriate reels and manage accounts.

Use-Case Modelling:

Each use case below belongs to the actor listed next to it.

Use case	**Actors**
Register Account	Business Owner, Registered Customer
Login	Business Owner, Registered Customer
Create Business Profile	Business Owner
Upload Reel	Business Owner, Cloudinary
Schedule Sale Reel	Business Owner
Delete Reel	Business Owner
Follow Business	Registered Customer
Like Reel	Registered Customer
Comment on Reel	Registered Customer, Business Owner
Send Message	Registered Customer, Business Owner
View Conversations	Registered Customer, Business Owner
Browse Feed	Registered Customer, Cloudinary
View Business Profile	Registered Customer
Expire Ended Sale	Scheduler

Scenario Analysis: Schedule a Sale Reel:

Role: Business Owner

Starting situation: A business owner has logged in and wants to announce a one-week sale that starts tomorrow. They open the upload screen and record a short video of the discounted product.

Normal Flow: The owner picks the video, writes a caption, the product name and the price, and chooses the type Sale. The owner sets the start date to tomorrow and the end date to seven days later. The app uploads the video to Cloudinary and sends the returned link and the dates to the API. The system saves the reel. It stays hidden until the start date, then appears in the feed of every customer who follows the business, and disappears when the end date passes.

Error Flow: If the end date is not after the start date, the system shows a message and does not save the reel. If the upload fails because of a weak connection, the app shows an error and a retry button. If the video is larger than the allowed size, the app asks the owner to choose a shorter video.

Alternative Flow: The owner can choose the type Regular, which has no dates and stays visible all the time. The owner can also delete a sale reel before it ends.

Other Activities: While the sale is running, customers keep watching and following the business, and the scheduler checks every minute and marks ended sales as expired.

Description of key use case:

System Name: Reels App

Use Case Title	**Schedule Sale Reel**
Description	The owner uploads a video and gives it a start date and an end date, so the reel is shown to followers only during the sale.
Actors	Business Owner, Cloudinary
Data	Video, Caption, Product name, Price, Start date, End date
Stimulus/Trigger	The owner opens the upload screen and chooses the type Sale.
Pre-Conditions	1) The owner is logged in and has a business profile. 2) The phone is connected to the internet.
Workflow or Sequence/Flow of Events	1) The system shows the upload form. 2) The owner picks a video and fills in the details. 3) The owner sets the start and end dates. 4) The app uploads the video to Cloudinary. 5) The app sends the video link and details to the API. 6) The system checks that the end date is after the start date. 7) The system saves the reel.
Post-Conditions / Response	1) The reel is saved with type Sale. 2) The reel is shown to followers from the start date until the end date.

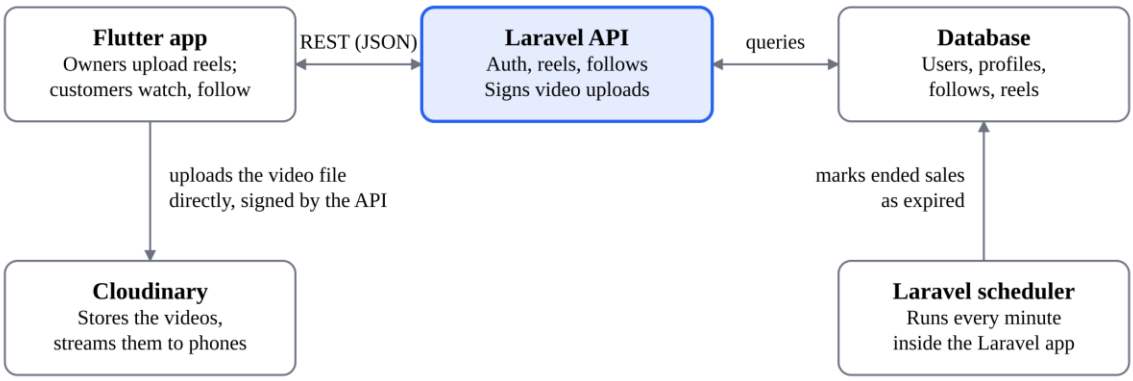
Tech stack:

Flutter on the front, Laravel on the back, and Cloudinary for video keep the project small enough to finish.

Layer	Choice	Why
Mobile app	Flutter	One codebase for Android and iOS
Backend	Laravel REST API	Fast to build, strong database and scheduling tools
Authentication	Laravel Sanctum	Token login designed for mobile apps
Database	MySQL	The data is relational: users, follows, reels
Video storage	Cloudinary	Stores videos, serves streaming URLs and thumbnails
Background jobs	Laravel Scheduler	Cleans up ended sales on a timer
App state	Riverpod or Bloc	Keeps UI separate from data calls

System architecture:

Video goes straight to Cloudinary; everything else goes through Laravel



The app sends video straight to Cloudinary, so large files never pass through Laravel; the API only signs the upload and stores the returned video URL.

Data model:

Eight tables carry the MVP: users, business profiles, follows, reels, likes, comments, conversations, and messages.

Table	Fields	Notes
users	id, name, email, password, role (owner or customer), avatar	One table for both roles
business_profiles	id, user_id, business_name, bio, phone or WhatsApp	Only for owners; one per owner
follows	follower_id, business_id	Unique pair, so no duplicate follows
reels	id, business_id, video_url, thumbnail_url, caption, product_name, price, type (regular or sale), starts_at, ends_at, created_at	ends_at is empty for regular reels
likes	user_id, reel_id	Unique pair, so one like per user
comments	id, reel_id, user_id, body, created_at	Body limited to 300 characters
conversations	id, customer_id, business_id	One conversation per customer and business
messages	id, conversation_id, sender_id, body, created_at	Text only in the MVP

Relations: a user has one business profile (owners only), a business has many reels, users follow businesses through the follows table, users like and comment on reels, and each conversation has many messages between one customer and one business.

Sale reel logic:

The feed decides what is visible, so a sale disappears at the exact second it ends even if the scheduler is late.

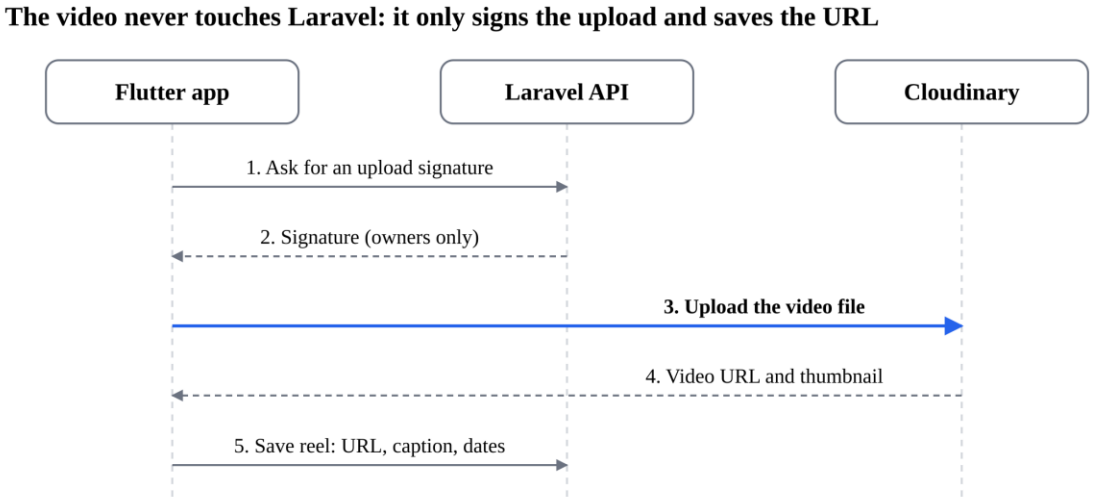
A reel appears in the feed only in one of two cases: it is a regular reel, or it is a sale reel and the current time is after its start date and before its end date.

Scheduling ahead: an owner publishes a sale reel today and sets the start date to tomorrow; it stays hidden until then.

Scheduler role: Laravel's scheduler runs every minute and marks ended sales as expired, so owners can see them in an "ended sales" list. The feed never depends on it.

Validation: the API rejects a sale reel when the end date is not after the start date, or when either date is missing.

Video upload flow:



Only steps 1, 2 and 5 touch Laravel; step 3 sends the large file straight to Cloudinary, which keeps uploads fast and the server light.